

Dark Pattern Detector

Detecting Manipulative Text on Indian E-Commerce Websites using NLP

Submitted by: Abhishek Joshi & Shaurya Ghorpade

Course: Artificial Intelligence | Date: 30 September 2026

Abstract. Dark patterns are pieces of interface text designed to pressure shoppers, such as fake countdown timers and false stock warnings. We fine-tuned DistilBERT to label each line of a product page as either not a dark pattern or one of five dark pattern types. On a benchmark of 2,356 e-commerce texts, split so that the test set contains only websites the model never saw, it reaches 96.6% F1 and names the type correctly 96.2% of the time. Because the benchmark comes from Western sites, we also evaluated the whole system on hand-labelled Amazon.in, Snapdeal and Indian brand-store pages. Better page parsing, a calibrated decision threshold and retraining on mistakes mined from real pages cut false alarms from 42.8 to 0 per page on held-out test pages, and on a live Amazon.in phone page flags fell from 92 to 2, both real dark patterns. The model powers a web dashboard that scans any product page from its link.

1. Introduction and Problem Statement

Dark patterns are interface designs that push people into decisions they did not intend to make. On shopping sites they are often just a line of text: a countdown ("Deal ends in 04:59"), a stock warning ("Only 2 left!"), a crowd claim ("27 people bought this in the last hour") or a guilt-trip opt-out ("No thanks, I don't like saving money"). A crawl of 11,000 shopping sites found 1,818 such instances (Mathur et al., 2019), and India's Central Consumer Protection Authority banned 13 dark patterns in its **Guidelines for the Prevention and Regulation of Dark Patterns, 2023**. Checking pages by hand does not scale.

Goal: build a tool that reads a product page and points out each manipulative line and its type, accurately enough to use on real Indian shopping sites, not only on a clean benchmark.

2. Data

Benchmark. We used the e-commerce dark pattern dataset of Yada et al. (2022), built from the Mathur et al. crawl: 2,356 text snippets from real shopping sites, half dark patterns and half ordinary text (Table 1).

Type	What it does	Example from the dataset	Texts
Scarcity	Claims stock is running out or demand is high	"Only 2 left in stock!"	418
Urgency	Puts a real or fake deadline on the decision	"Deal ends in 04:59"	210
Social Proof	Uses other shoppers' activity as pressure	"27 people bought this in the last hour"	312
Misdirection	Guilt-trips or steers the choice	"No thanks, I don't like saving money"	195
Other	Obstruction, sneaking and forced action (merged: too few examples)		43
Not a dark pattern	Ordinary shop text	"Write a review"	1,178

Table 1: Dark pattern types in the benchmark.

Indian shopping text. The benchmark comes from Western sites, so we added (a) 368 Indian-style training lines generated from templates (rupee prices, delivery offers, ratings, product specifications), plus 32 separately written held-out lines; and (b) **171 lines mined from 17 real product pages** on Amazon.in, Snapdeal and six Indian brand stores: every line the first model scored above 0.3 or that used dark-pattern keywords, labelled by hand (11 dark, 160 not).

Real-page evaluation. We labelled every line of 16 more product pages: 5 dev pages for tuning and 5 test pages from Amazon.in and Snapdeal (9 dark lines), plus 6 pages from **brand stores never used in training or tuning** (3 dark lines). None of these pages were used for mining.

3. Method

Baselines. TF-IDF vectors (unigrams and bigrams) with Logistic Regression and a Linear SVM.

Model. DistilBERT (Sanh et al., 2019), a compressed BERT that keeps about 97% of its language understanding at 40% smaller size, reads each word in context: "only" is pressure in "only 3 left" but harmless in "only available in cotton". Our first model had 2 classes plus a separate TF-IDF type classifier. The final model has **one head with 6 classes**, not a dark pattern plus the five types, so it can say "none of these" and its two decisions cannot disagree. $P(\text{dark})$ is 1 minus $P(\text{not a dark pattern})$. Training: 3 epochs, AdamW, learning rate $2e-5$, batch 16, 64 tokens.

Honest splits. A random split puts text from the same website in both training and test. The final benchmark split keeps each website on one side (GroupShuffleSplit on page id), so test scores measure unseen sites.

From a page to lines. The server downloads the page and keeps the visible text of each block element (paragraph, list item, table cell) as one line, joining inline tags so sentences are not split into fragments. It skips hidden text, dropdown lists, template placeholders, customer reviews (written by shoppers, not the seller) and lines with fewer than two words. On the Amazon.in phone page mentioned in the abstract, this cut 690 messy lines to 164 clean ones.

Decision threshold. Only 1 to 2% of the lines on a real page are dark patterns, against 50% in training, so a 0.5 cut-off raises far too many false alarms. A line is flagged when $P(\text{dark}) > 0.97$, chosen on the dev pages.

4. Results

Model	Test split	Accuracy	Precision	Recall	F1
TF-IDF + Logistic Regression	Random	95.6%	94.6%	96.6%	95.6%
TF-IDF + Linear SVM	Random	95.1%	94.2%	96.2%	95.2%
DistilBERT, 2 classes (first model)	Random	96.8%	96.6%	97.0%	96.8%
DistilBERT, 6 classes (final)	By website	96.6%	97.0%	96.2%	96.6%

Table 2: Dark pattern vs. not on the benchmark. The first three rows use a random split; the final model is tested on unseen websites.

The final model reaches 96.6% F1 on websites it never saw and names the type correctly 96.2% of the time (macro F1 0.935, up from 0.894 without the mined lines). It gets all 32 held-out Indian lines right. Trained and tested on a random split instead, the same setup scores 96.3%, so the random split did not inflate the earlier numbers.

Real product pages. Table 3 follows the full pipeline, from page to verdict, through each change.

Pipeline	Test F1	Test found	Test false alarms/page	Unseen F1	Unseen found	Unseen false alarms/page
First model, original page parser, threshold 0.5	0.11	9 of 9	42.8	0.06	2 of 3	10.3
+ new page parser	0.33	9 of 9	7.2	0.11	2 of 3	5.3
+ threshold 0.97	0.50	6 of 9	1.8	0.22	2 of 3	2.2
6-class model, no mined data	0.80	6 of 9	0.0	0.80	2 of 3	0.0
6-class model + mined data (final)	1.00	9 of 9	0.0	0.80	2 of 3	0.0

Table 3: Real-page results. "Found" counts labelled dark lines that were flagged; false alarms are flagged lines that are not dark patterns.

Page parsing removed most false alarms, the threshold removed most of the rest, and the 6-class model and mined data raised recall on the test pages back to every dark pattern. On the Amazon.in page that started this work, flags fell from 92 to 2 ("Order within 8 hrs 40 mins" and "100K+ recent orders from this brand"). **Caveats:** the evaluation sets are small (12 dark lines in total); the 0.97 threshold was kept, rather than re-tuned on dev pages that share sites with the mined data, after seeing the unseen-site results, so those results are optimistic; and the labels follow our written guide, which counts purchase counts such as "recent orders" as Social Proof.

5. System

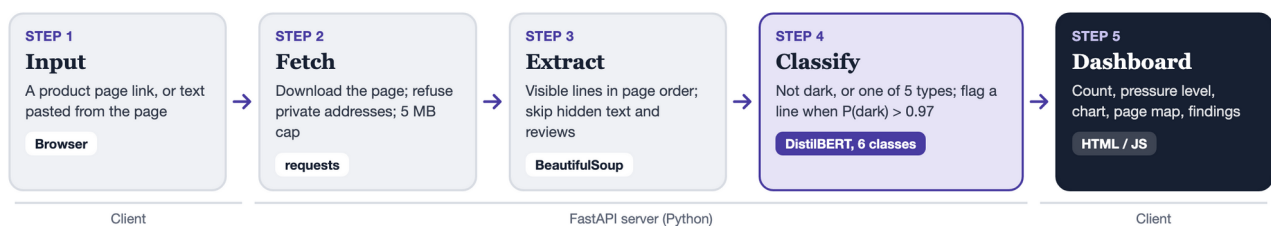


Figure 1: System architecture and data flow for one scan.

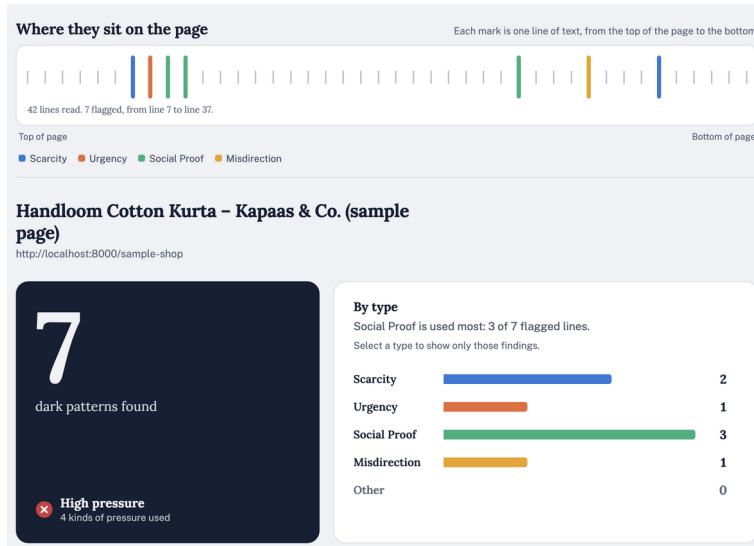


Figure 2: The dashboard: where each dark pattern sits on the page, the verdict, and a chart by type.

The dashboard states the main result in words, marks where each dark pattern sits on the page, explains why each line was flagged with its closest CCPA 2023 category, and lets people mark a mistake as "Not a dark pattern" (with Undo). It follows the system light or dark appearance, meets the WCAG AA contrast minimum of 4.5:1 for text, supports larger text, reduced motion and keyboard use, and was audited against Apple's Human Interface Guidelines. It is built with PyTorch and Hugging Face Transformers (training), FastAPI, requests and BeautifulSoup (server) and plain HTML, CSS and JavaScript (dashboard), and ships as a Docker image. When hosted publicly, the server refuses links to private network addresses.

To run it:

```
pip install -r app/requirements.txt
python app/train.py # trains the model (about 5 minutes on a laptop GPU)
python app/server.py # then open http://localhost:8000
```

6. Limitations and future work

- **Text only.** Visual tricks such as pre-ticked boxes, tiny decline buttons and hidden costs need a vision model trained on page screenshots.
- **Wording, not truth.** The model flags pressure language; it cannot tell whether "Only 2 left" is false.
- **Small real-page sets.** A few hundred hand-labelled Indian pages, including Hinglish and regional languages, would give firmer numbers.
- **Page coverage.** Sites that block bots or render with JavaScript need pasted text; a browser extension would read the page as the shopper sees it.

7. Conclusion

A 6-class DistilBERT model detects dark patterns in shopping text with 96.6% F1 on unseen websites. Testing on real Indian product pages showed that benchmark accuracy was not enough: page parsing, a threshold matched to real pages and retraining on real mistakes took the system from 42.8 false alarms per page to none on the test pages. The dashboard makes the result usable by shoppers, and by regulators enforcing the CCPA 2023 guidelines.

References

- [1] Mathur, A., Acar, G., Friedman, M. J., Lucherini, E., Mayer, J., Chetty, M., & Narayanan, A. (2019). Dark Patterns at Scale: Findings from a Crawl of 11K Shopping Websites. Proc. ACM on Human-Computer Interaction (CSCW).
- [2] Yada, Y., Feng, J., Matsumoto, T., Fukushima, N., Kido, F., & Yamana, H. (2022). Dark Patterns in E-Commerce: A Dataset and Its Baseline Evaluations. IEEE International Conference on Big Data.
- [3] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
- [4] Central Consumer Protection Authority, Government of India (2023). Guidelines for the Prevention and Regulation of Dark Patterns, 2023.

Source code: <https://github.com/ketanabhishek8/Dark-Pattern-Website-Detector>